

# Week 1 Code Lab - Complete Solutions

Environment Setup and Your First Agent/Problem Skeleton

CMPSC 480 - Instructor Solution Deck

**Instructor material - do not distribute with the question deck**

# Solution overview

---

**Answer.** The reference implementation uses:

- ▶ a tuple  $(row, column)$  as the immutable state;
- ▶ an Enum for the four actions;
- ▶ a validated GridProblem with deterministic action order;
- ▶ an abstract generic Agent contract;
- ▶ unittest for normal, boundary, illegal, and malformed cases; and
- ▶ a verification script that fails with a nonzero process status.

Equivalent designs earn full credit when they satisfy the same observable contract.

# Reference repository structure

---

## Complete submission

```
week01-agent-skeleton/  
  README.md  
  verify_environment.py  
  src/  
    __init__.py  
    agent.py  
    problem.py  
  tests/  
    test_problem.py
```

**Grading guidance.** Accept a `pyproject.toml` or package-based structure if imports and the documented test command work from a clean checkout.

# Solution 1 - environment verification imports

---

```
from __future__ import annotations

import subprocess
import sys
from importlib.metadata import PackageNotFoundError, version

def package_version(name: str) -> str:
    try:
        return version(name)
    except PackageNotFoundError as exc:
        raise RuntimeError(
            f"required package is not installed: {name}"
        ) from exc
```

# Solution 1 - Git command helper

---

```
def git_output(*args: str) -> str:
    completed = subprocess.run(
        ["git", *args],
        check=True,
        capture_output=True,
        text=True,
    )
    return completed.stdout.strip()
```

**Answer.** `check=True` converts a failed Git command into a nonzero program outcome instead of silently accepting empty output.

# Solution 1 - version and package checks

---

```
def main() -> None:
    if sys.version_info < (3, 10):
        raise RuntimeError("Python 3.10 or newer is required")

    print(f"PYTHON={sys.version.split()[0]}")
    print(f"NUMPY={package_version('numpy')}")
    print(f"JUPYTER={package_version('jupyterlab')}")
```

**Answer.** Stable labels make the output readable by both a person and a simple grading script.

# Solution 1 - Git checks and entry point

---

```
# Continue inside main().
if git_output("rev-parse", "--is-inside-work-tree") != "true":
    raise RuntimeError("current directory is not a Git work tree")

commits = int(git_output("rev-list", "--count", "HEAD"))
if commits < 1:
    raise RuntimeError("the repository has no commits")
print(f"GIT_COMMITS={commits}")

if __name__ == "__main__":
    main()
```

# Solution 1 - Expected behavior (2 points)

---

## Representative successful output

```
PYTHON=3.11.0  
NUMPY=2.1.0  
JUPYTER=4.2.0  
GIT_COMMITS=3
```

- ▶ Exact installed versions may differ.
- ▶ Missing software, an old Python version, a nonrepository directory, or zero commits must produce a nonzero exit status.

**Grading guidance.** Award full credit when the output is stable and every required condition affects process success.

# Solution 2 - immutable state and action model

---

```
from __future__ import annotations

from enum import Enum
from typing import TypeAlias

State: TypeAlias = tuple[int, int]

class Action(Enum):
    UP = (-1, 0)
    DOWN = (1, 0)
    LEFT = (0, -1)
    RIGHT = (0, 1)
```

**Answer.** A two-integer tuple has value equality, stable hashing, and no mutating methods. It can therefore serve as a set member or dictionary key.

# Solution 2 - class and constructor signature

---

```
class GridProblem:
    ACTION_ORDER = (
        Action.UP, Action.DOWN, Action.LEFT, Action.RIGHT
    )

    def __init__(
        self,
        rows: int,
        columns: int,
        start: State,
        goal: State,
        blocked: frozenset[State] = frozenset(),
    ) -> None:
```

**Answer.** A fixed action order makes successor generation deterministic, which simplifies testing and debugging.

# Solution 2 - constructor state

---

```
# Continue inside GridProblem.__init__.  
if rows < 1 or columns < 1:  
    raise ValueError("grid dimensions must be positive")  
self.rows = rows  
self.columns = columns  
self.start = start  
self.goal = goal  
self.blocked = frozenset(blocked)
```

**Answer.** Copying blocked into a frozenset prevents later caller mutation from changing the problem definition.

# Solution 2 - constructor domain validation

---

```
# Continue inside GridProblem.__init__.
self._validate_state(start)
self._validate_state(goal)
for state in self.blocked:
    self._validate_coordinate(state)
if start in self.blocked or goal in self.blocked:
    raise ValueError("start and goal must be traversable")
```

**Answer.** Every configured coordinate is checked once at the public boundary.

# Solution 2 - state-validation helpers

---

```
def _validate_coordinate(self, state: State) -> None:
    if (
        not isinstance(state, tuple)
        or len(state) != 2
        or any(type(value) is not int for value in state)
    ):
        raise TypeError("state must be a tuple of two integers")
    row, column = state
    if not (0 <= row < self.rows and 0 <= column < self.columns):
        raise ValueError(f"state is outside the grid: {state}")

def _validate_state(self, state: State) -> None:
    self._validate_coordinate(state)
    if state in self.blocked:
        raise ValueError(f"state is blocked: {state}")

def initial_state(self) -> State:
    return self.start
```

# Solution 2 - legal actions

---

```
def _destination(self, state: State, action: Action) -> State:
    delta_row, delta_column = action.value
    return state[0] + delta_row, state[1] + delta_column

def actions(self, state: State) -> list[Action]:
    self._validate_state(state)
    legal: list[Action] = []
    for action in self.ACTION_ORDER:
        candidate = self._destination(state, action)
        try:
            self._validate_state(candidate)
        except ValueError:
            continue
        legal.append(action)
    return legal
```

**Answer.** Iterating through ACTION\_ORDER makes tie behavior deterministic and testable.

# Solution 2 - validated transition

---

```
def result(self, state: State, action: Action) -> State:
    self._validate_state(state)
    if not isinstance(action, Action):
        raise TypeError("action must be an Action value")
    if action not in self.actions(state):
        raise ValueError(f"illegal action {action.name} at {state}")
    return self._destination(state, action)
```

**Answer.** The returned tuple is a new value. The caller's state remains unchanged and safe as an explored-set key.

# Solution 2 - validated goal test

---

```
def goal_test(self, state: State) -> bool:
    self._validate_state(state)
    return state == self.goal
```

**Answer.** Validation prevents a malformed coordinate from silently becoming a normal negative result.

# Solution 2 - verified step cost

---

```
def step_cost(
    self,
    state: State,
    action: Action,
    next_state: State,
) -> float:
    expected = self.result(state, action)
    self._validate_state(next_state)
    if next_state != expected:
        raise ValueError(
            f"{next_state} is not the result of {action.name} at {state}"
        )
    return 1.0
```

**Answer.** Reusing result centralizes the legal-transition contract and prevents cost calculation for an impossible edge.

# Solution 3 - Agent contract (1 point)

---

```
from abc import ABC, abstractmethod
from typing import Generic, TypeVar

PerceptT = TypeVar("PerceptT")
ActionT = TypeVar("ActionT")

class Agent(ABC, Generic[PerceptT, ActionT]):
    """Base interface for agents that map percepts to actions."""

    @abstractmethod
    def act(self, percept: PerceptT) -> ActionT:
        """Return the next action for the supplied percept."""
        raise NotImplementedError
```

**Answer.** The abstract method makes incomplete use fail immediately while preserving a stable Week 2 extension point.

# Solution 4 - Contract tests and repository quality (2 points)

---

```
import unittest

from src.problem import Action, GridProblem

class GridProblemTests(unittest.TestCase):
    def setUp(self) -> None:
        self.problem = GridProblem(
            rows=3,
            columns=3,
            start=(0, 0),
            goal=(2, 2),
            blocked=frozenset({(1, 1)}),
        )
```

**Answer.** A shared fixture keeps every test focused on one behavior.

# Solution 4 - normal and boundary tests

---

```
def test_corner_actions_are_deterministic(self) -> None:
    self.assertEqual(
        self.problem.actions((0, 0)),
        [Action.DOWN, Action.RIGHT],
    )

def test_known_transition(self) -> None:
    self.assertEqual(
        self.problem.result((0, 0), Action.RIGHT),
        (0, 1),
    )
```

**Grading guidance.** Award one point for focused behavioral tests and one point for reproducible documentation and repository quality.

# Solution 4 - illegal, goal, and hashing tests

---

```
def test_illegal_action_is_rejected(self) -> None:
    with self.assertRaises(ValueError):
        self.problem.result((0, 0), Action.UP)

def test_goal_test_and_hashability(self) -> None:
    self.assertFalse(self.problem.goal_test((0, 0)))
    self.assertTrue(self.problem.goal_test((2, 2)))
    explored = {(0, 0), (0, 1)}
    self.assertIn((0, 1), explored)
```

# Solution 4 - degenerate and malformed cases

---

```
def test_one_cell_grid_has_no_actions(self) -> None:
    problem = GridProblem(1, 1, (0, 0), (0, 0))
    self.assertEqual(problem.actions((0, 0)), [])

def test_malformed_state_is_rejected(self) -> None:
    with self.assertRaises(ValueError):
        self.problem.goal_test((3, 0))

if __name__ == "__main__":
    unittest.main()
```

# Edge-case reasoning

---

## One-cell grid

The state may be both initial and goal. The legal-action list is empty, which is distinct from a failure value.

## Illegal action

Returning the unchanged state would hide a caller defect. The reference contract raises `ValueError`.

## Blocked cells

Configuration may name blocked coordinates, but no public operation may accept one as a traversable state.

## Stable hashing

Every transition creates a new tuple. Existing state keys remain valid in sets and dictionaries.

# Rubric solution - Correctness (8 points)

---

| Category                      | Points | Full-credit evidence                                                                  |
|-------------------------------|--------|---------------------------------------------------------------------------------------|
| Environment verification      | 2      | All versions, Git work-tree status, positive commit count, and nonzero failure status |
| Immutable state model         | 1      | Stable equality and hashing; no mutation                                              |
| Legal actions and transitions | 2      | Correct deterministic behavior at boundaries and blocked cells                        |
| Goal test and step cost       | 1      | Boolean goal result; positive cost only for the matching legal transition             |
| Defensive validation          | 1      | Precise type/domain/action rejection                                                  |
| Agent abstraction             | 1      | Abstract, documented, type-parameterized contract                                     |

# Rubric solution - Tests and quality (2 points)

---

| Category                     | Points | Full-credit evidence                                                                                               |
|------------------------------|--------|--------------------------------------------------------------------------------------------------------------------|
| Focused tests                | 1      | Normal transition, boundary actions, illegal action, goal cases, hashability, no-action state, and malformed state |
| Documentation and repository | 1      | Public docstrings, complete type hints, reproducible README, clean tree, and meaningful commit                     |
| Total                        | 10     |                                                                                                                    |

**Grading guidance.** Deduct only once for one underlying defect that causes several downstream test failures.

# Reflection exemplar

---

**Answer.** The initial-state component maps to the immutable tuple returned by `initial_state`. The action function maps to `actions(state)`, whose deterministic list contains exactly the moves that stay in bounds and avoid blocked cells. The transition model maps to `result`, which validates the current state and action before creating a new tuple. The goal test compares a validated state with the configured goal, and the path-cost model assigns 1.0 to each verified legal transition.

One ambiguity was whether an illegal move should leave the state unchanged. I chose an exception because the method contract accepts only legal actions; silently staying in place would hide a caller defect. A tuple state has value equality and stable hashing, so Week 2 can store it safely in an explored set.

# Instructor verification commands

---

Run from a clean checkout:

## Environment

```
python verify_environment.py
```

## Tests

```
python -m unittest discover -s tests -v
```

## Repository evidence

```
git status -short  
git log -1 -oneline
```

**Key idea.** The reference implementation contains no search algorithm; Week 1 establishes the abstraction that Week 2 will extend.